

GROUP 10

Shrenik Jain(24b2302)

Aayush Shah(24b2300)

QUESTION 1.

1. $x(k+1) = x(k) + w(k)$

$$\begin{bmatrix} y_1(k) \\ y_2(k) \end{bmatrix} = \begin{bmatrix} 2 \\ \ln k \end{bmatrix} x(k) + \begin{bmatrix} v_1(k) \\ v_2(k) \end{bmatrix}$$

Since the system is a linear time variant-system, the Extended-Kalman filter (EKF) is identical to the standard Kalman filter (KF).

STATE-ESTIMATION-EQUATIONS

① State - Prediction

$$\hat{x}_{k|k-1} = A \hat{x}_{k-1|k-1}$$

$$A_{k-1} = A = 1$$

Assumption:
 $P_{0|0} = 1$

② Covariance - Prediction

$$P_{k|k-1} = A P_{k-1|k-1} A^T + Q_{k-1}$$

$$Q_{k-1} = Q = 0.05$$

$$\Rightarrow P_{k|k-1} = P_{k-1|k-1} + 0.05$$

③ Kalman gain (K_k)

$$K_k = P_{k|k-1} \cdot H_k^T \cdot (H_k P_{k|k-1} H_k^T + R_k)^{-1}$$

$$\therefore H_k = \begin{bmatrix} 2 \\ \ln k \end{bmatrix}$$

$$R_k = \begin{bmatrix} 1 & 0 \\ 0 & \ln k \end{bmatrix}$$

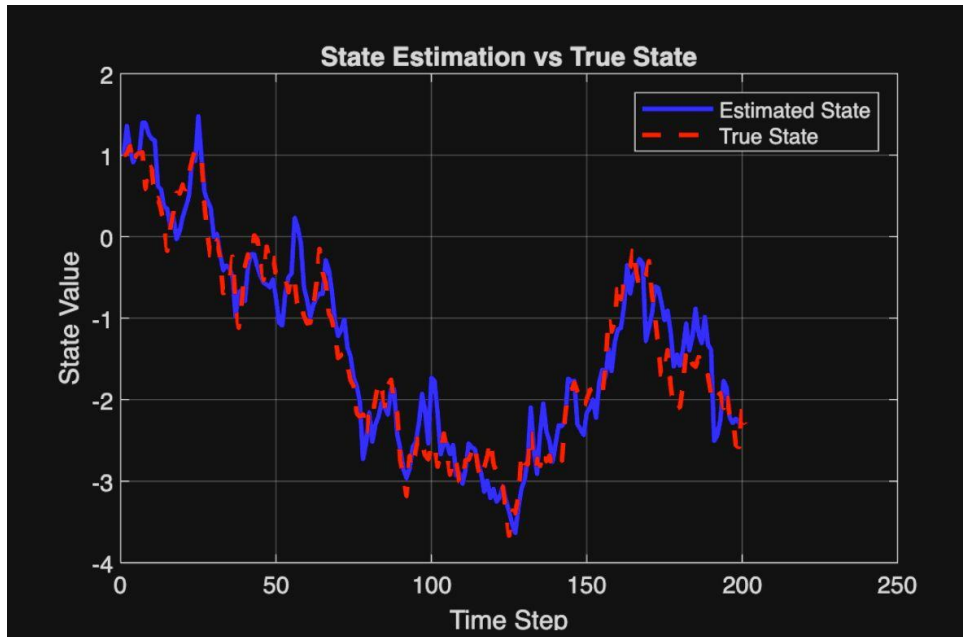
④ State - Update

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k \left(y(k) - H_k \hat{x}_{k|k-1} \right)$$

$$\hat{y}(k) = H_k \cdot \hat{x}_{k|k-1}$$

⑤ Covariance - Update

$$P_{k|k} = (I - K_k H_k) \cdot P_{k|k-1}$$



Interpretation of the Results and Plot

The plot compares the True State (red dashed line) with the EKF Estimated State (solid blue line) over the simulation horizon. The Extended Kalman Filter exhibits effective performance, as explained below.

(a) Good Tracking Performance

The estimated state follows the true state closely for the majority of the time steps. This demonstrates that the Extended Kalman Filter is successfully integrating information from:

- the state prediction equation ($\hat{x}_{k|k-1}$) and covariance prediction equation ($P_{k|k-1}$)
- the noisy measurement signals

to produce an accurate estimate of the underlying system state. The filter effectively reduces uncertainty and ensures that the estimate adapts to changes in the observed data.

(b) Occasional Deviations

Minor deviations between the true and estimated states appear at some points. These deviations typically occur when:

- the measurement noise increases, particularly when our calculation of the noise using the normal distribution gives us very random value.
- the system experiences relatively sharper or quicker changes

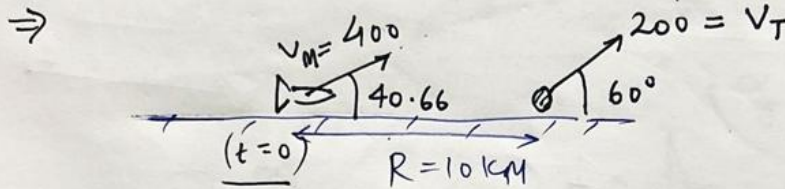
QUESTION 2.

2. At initial moment, on collision course. -

$\dot{\theta}$ must be 0, $\therefore 400 \sin(\alpha - 15^\circ) = 200 \sin 60^\circ$

On solving this eqⁿ, [where $15^\circ =$ initial heading error]

$\therefore \alpha = 40.66^\circ$ (launch angle of missile)



$V_{r_x} = \dot{r}_x = -203.44 \text{ ms}^{-1}$
 $\therefore \text{at } t = 3, R_x = \left(10 + \dot{r}_x \times \frac{3}{1000}\right)$

$\Delta x = R_x = 9.39 \text{ km}$

$\Delta y = (400 \times \sin(40.66) - 200 \sin 60^\circ) \times 3$

$\Delta y = 260.1 \text{ m}$

$\Rightarrow \Delta R = \sqrt{\Delta x^2 + \Delta y^2}$

$\Rightarrow \Delta R = \sqrt{8.8202 \times 10^7} = 9395 \text{ m}$

For $t > 3$

We know for PPN guidance,

$$a_m = N V_M \dot{\theta} \quad \& \quad \dot{\alpha}_M = N \dot{\theta}$$

For PN guidance,

optimal value of N is 3
(which is obtained by minimizing

$\int a_m^2(t) dt$ while a constraint that interception occurs).

$$\text{Also, } a_m = f(z) = \frac{V_M \Delta \phi_i N}{t_f} \left(\frac{z}{t_f} \right)^{N-2}$$

for any Proportional Navigation with a heading error.

$$t_f = \frac{y(3)}{V_c} = \frac{9395}{-V_h} ;$$

$$\theta = \tan^{-1} \left(\frac{\Delta y}{\Delta x} \right) = -1.588^\circ$$

$$\Rightarrow V_h = V_t \cos(68.588^\circ) - V_M \cos(42.188^\circ)$$

$$|V_h = -200.8$$

$$\therefore t_f = 46.78 \text{ sec.}$$

$$\therefore a_m(3) = V_M \Delta \theta$$

$a_m(t_f)$ = lateral acceleration when PPN starts.

$$a_m(t_f) = \frac{V_M \Delta \phi \cdot N}{t_f} \cdot 1$$

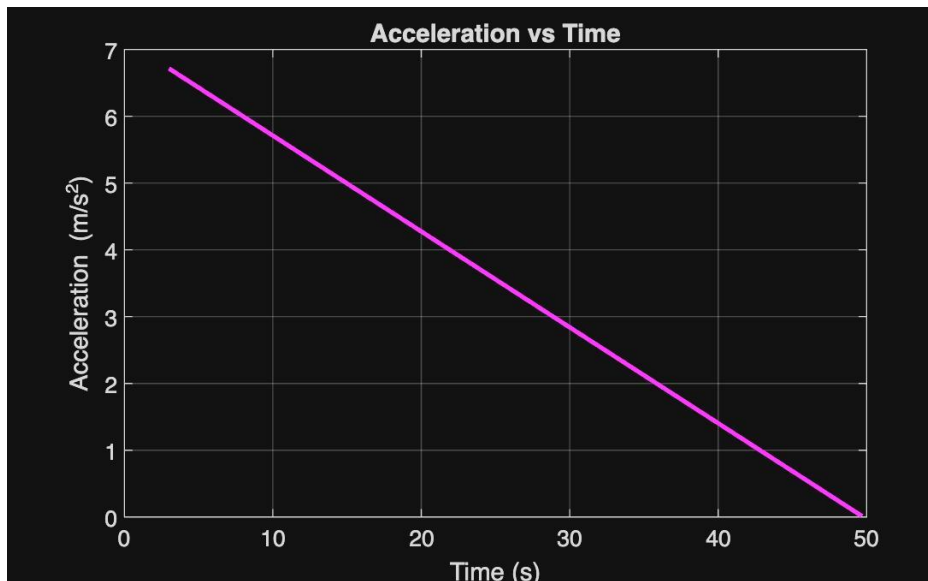
$$= \frac{400 \cdot 15 \cdot 3 \cdot \pi}{46.78 \cdot 180}$$

$$= 384.78 \text{ ms}^{-2}$$

$$a_m(t_f) = 6.7 \text{ ms}^{-2}$$

$$(b) \text{ for } N=3 \quad a_m = \frac{V_M \Delta \phi_i N}{t_f} \left(\frac{z}{t_f} \right)^1$$

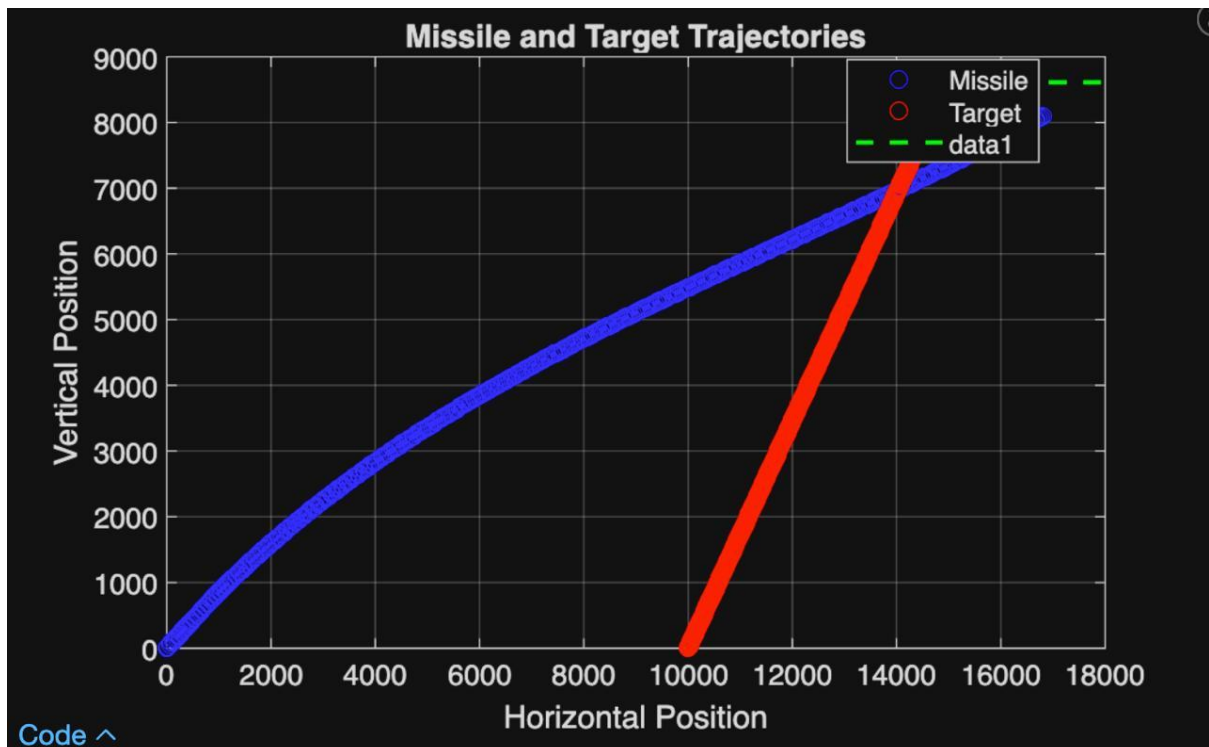
$$\text{also, } \Delta \phi(z) = \Delta \phi_i \left(\frac{z}{t_f} \right)^{N-1}$$



OBSERVATIONS :

The lateral acceleration plot demonstrates the key property of PPN guidance:

- High acceleration early to correct large initial deviation errors.
- Lower acceleration later as the missile settles onto the proper collision course.



OBSERVATIONS :

- Overshoot is due to a Miscalculation of instantaneous deviation caused by:
 - (a) Incorrect small angle approximation while calculating the formula.
 - (b) Assumption that tf is constant.

Question 1

```
mu = 0;
n = 200;
sigma = sqrt(0.05);
noise = mu + sigma * randn(1, n);
v1 = mu + sqrt(1) * randn(1, n);

for k = 1:n
    v2(k) = sqrt(abs(cos(k))) * randn(1);
end

y = zeros(2, n);
x_est = zeros(1, n);
x_true(1) = 1;

y(:, 1) = [0 0]';
y(:, 2) = [3 1]';

for k = 1:length(noise)
    x_true(k+1) = x_true(k) + noise(k);
end

for k = 3:n
    H = [2 ; sin(k-1)];
    y(:, k) = H * x_true(k) + [v1(k) v2(k)]';
end

x_est(1) = 1;
cov(1) = 1;
for k = 1:length(noise)-1
    H = [2 sin(k)]';
    R = [1 0; 0 abs(cos(k))];
    x_est_pred = x_est(k); % State Prediction
    cov_pred = cov(k) + 0.05; % Covariance Prediction
    kalman_gain = cov_pred * H' * inv(H * cov_pred * H' + R); % Kalman Gain
    x_est(k+1) = x_est_pred + kalman_gain * (y(:, k+1) - H * x_est_pred); % State Update
    cov(k+1) = (1 - kalman_gain * H) * cov_pred; % Covariance Update
end

figure;
plot(1:length(x_est), x_est, 'b-', 'LineWidth', 2); % Plot estimated state
hold on;
plot(1:length(x_true), x_true, 'r--', 'LineWidth', 2); % Plot true state
xlabel('Time Step');
ylabel('State Value');
title('State Estimation vs True State');
legend('Estimated State', 'True State');
grid on;
hold off;
```

Question 2

```
clear all
clc
Vm = 400;
Vt = 200;
alphan_ideal = 25.66 * pi/180;
alphan = 60 * pi / 180;
alphan = 40.66 * pi / 180;
ri = 10000;
N = 3;
tf = 46.78
i_deviation = 15 * pi / 180;

t_new = 3:0.1:tf + 3;
am = (Vm * 15 * (pi / 180) * N * ((tf + 3 - t_new) / tf) .^ (N - 2)) / tf;
figure;
plot(t_new, am, 'm-', 'LineWidth', 2);
xlabel('Time (s)');
ylabel('Acceleration (m/s^2)');
title('Acceleration vs Time');
grid on;

t = 0:0.1:3;
t1 = 0:0.1:tf+3
x_missile = Vm * cos(alphan_ideal + i_deviation) * t;
y_missile = Vm * sin(alphan_ideal + i_deviation) * t;
x_target = Vt * cos(alphan) * t1 + 10000;
y_target = Vt * sin(alphan) * t1;

deviation = i_deviation * (((tf - t_new + 3) / tf) .^ (N - 1));

for k = 1:length(t_new)
    x_missile(k + 31) = Vm * cos(alphan_ideal + deviation(k)) * t_new(k);
    y_missile(k + 31) = Vm * sin(alphan_ideal + deviation(k)) * t_new(k);
end

for k = 1:length(t_new)
    plot(x_missile(k), y_missile(k), 'bo');
    hold on;
    plot(x_target(k), y_target(k), 'ro');
    pause(0.1);
end
xlabel('Horizontal Position');
ylabel('Vertical Position');
title('Missile and Target Trajectories');
legend('Missile', 'Target');

end_point_missile = [x_missile(end), y_missile(end)];
end_point_target = [x_target(end), y_target(end)];
plot([end_point_missile(1), end_point_target(1)], [end_point_missile(2), end_point_target(2)], 'g--', 'LineWidth', 1.5);
grid on;
```